SENTINEL

Simulated ENtity Threat Intelligence & Engagement Layer

Project Overview & Architecture Document

Project Overview

SENTINEL is a full-stack simulation of an AI-assisted battlefield command and control interface, inspired by Palantir Gotham and Anduril's Lattice/EagleEye platforms. It demonstrates human-in-the-loop AI design applied to a tactical decision-making context — a genuinely active area of defense AI research and ethics.

The operator manages a live synthetic battlespace: scoring threats, issuing commands to friendly units, and making time-pressured decisions alongside an AI that can be wrong. A post-scenario debrief reveals outcomes and surfaces operator bias patterns.

High-Level Architecture

```
[Simulation Engine]  →  [Classifier]  →  [LLM Explainer]  →  [Operator Interface]
      Python                Python          Claude API             React 19
```

Entity Types

The simulation tracks four types of entities on the battlefield. Each has distinct statistical fingerprints that the classifier learns from synthetic training data.

Type	Color (UI)	Map Marker	Description
Hostile	Red #E24B4A	Red circle / icon	Enemy combatants — clearly threatening sensor fingerprint
Friendly	Blue #378ADD	Blue icon + heading arrow	Allied units — operator issues tactical commands to manage them
Civilian	Gray #888780	Gray circle	Non-combatants — misidentification has serious consequences
Ambiguous	Amber #EF9F27	Amber circle	Deliberately unclear — where AI/LLM disagreements are most frequent

The Four Components

1. Simulation Engine (Python)

Generates and continuously updates the synthetic battlefield. Each simulation tick produces:

- Entities moving on a map with lat/long coordinates, heading, and speed
- Sensor events attached to each entity — signals detected, proximity to known zones, pattern-of-life data

- Natural language intelligence reports for each entity (used by the LLM explainer)
- Scenario scripting controlling the ratio of hostile / friendly / civilian / ambiguous entities

Friendly units additionally carry:

- A command state (Hold, Fall Back, Advance & Clear, Request Status)
- An assigned target ID when directed toward a hostile
- A unit callsign (e.g. Alpha-1, Bravo-3) for operator identification
- A status field (Active, Advancing, Falling Back, Holding, Incapacitated)

2. Classifier (Python / scikit-learn)

Takes structured sensor features for each entity and outputs a scored threat assessment:

- threat_score — float from 0.0 (no threat) to 1.0 (confirmed hostile)
- confidence — how certain the model is of its score
- feature_importance — which signals drove the score (shown as a bar chart in the UI)

Training data is generated synthetically from the simulation engine itself. Hostile entities carry statistical fingerprints; civilian entities carry others; noise is added to blur the boundary and create realistic false positive rates.

3. LLM Explainer (Claude API)

When the operator clicks Explain on a target, the backend sends the entity's intelligence report and classifier outputs to Claude, which reasons through the evidence and returns a structured explanation:

- Summary of key evidence
- Step-by-step reasoning chain
- Confidence caveat
- Recommended operator action
- Civilian risk assessment (low / medium / high)

The critical design tension: the LLM and classifier will sometimes disagree — a high classifier score paired with a cautious LLM explanation, or vice versa. This disagreement is surfaced explicitly to the operator as an additional signal, and tracked in the post-scenario debrief.

4. Operator Interface (React 19)

A live tactical operations interface giving the operator full situational awareness and command capability. Built with React 19 + Vite + TypeScript, using a fixed three-panel layout.

Layout

```
┌──────────────────────────────────────────────────────────────────┐ │
Header                                │
├──────────────────────┬──────────────────────────────────────────┤ │
│                                      │ │    TargetQueue          │                MapView
│  │    (220px fixed)     │            (flex-1)                    │ │
│                                      │
├──────────────────────┴──────────────────────────────────────────┤ │
DossierPanel                                 │ │        (full width, slides up on entity
select)              │
└──────────────────────────────────────────────────────────────────┘
```

Map

- MapLibre GL JS map with Stadia Maps dark tile layer (alidade_smooth_dark) — no API key required
- Entity markers coloured by type using data-driven expressions — no conditional React rendering
- Icons update position in real time as entities move
- Friendly units render a heading arrow layer showing current direction of travel
- Selected entity marker enlarged via a match expression on feature ID

Target Queue

Scrollable left sidebar (220px fixed) showing all active entities sorted by threat score descending. Always visible — time pressure mechanic requires countdown timers in view at all times.

- Each row: entity type icon, ID or callsign, threat score badge, status badge, countdown timer
- Left border accent colour matches entity type — red (hostile), amber (ambiguous), blue (friendly), gray (civilian)
- Filter buttons for entity type; toggle sort by threat score or time remaining

Per-Target Dossier

Full-width panel that slides up from the bottom (Framer Motion) when an entity is selected. Three sub-components rendered side-by-side:

- ClassifierResult — threat score progress bar, confidence bar, feature importance horizontal BarChart (Recharts)
- LLMExplainer — Explain button, reasoning chain as numbered steps, civilian risk badge, recommended action badge, disagreement indicator
- ActionBar — conditional buttons by entity type (see below)

Action Bar

Hostile / Ambiguous / Civilian entities:

  - **Approve Engagement**
  - **Deny Engagement**
  - **Request More Intel**

Friendly entities:

  - **Fall Back** — move unit away from nearest threat
  - **Advance & Clear** — direct unit toward an assigned hostile target
  - **Hold Position** — keep unit stationary
  - **Request Status Update** — ping unit for current condition report

Operational Mechanics

- Countdown timer per target to simulate time pressure — flashes red under 10 seconds
- Friendly units update position in real time based on issued commands
- Emergent scenarios: friendlies approaching ambiguous targets, converging on hostiles, etc.

Emergent Scenarios

The combination of friendly command mechanics and ambiguous entities creates tactically interesting situations the operator must navigate:

- A friendly unit closes on an ambiguous entity — resolve the target first, or pull the friendly back?
- Mid-movement sensor data upgrades a target's threat score — abort the advance or continue?

- Two friendlies converge on the same hostile from different directions — coordination challenge
- Classifier flags a friendly as a threat — surfaced as a false positive in the debrief
- A civilian is caught between a friendly advance and a hostile position

Data Flow — Single Target Lifecycle

1. Simulation spawns entity
2. Sensor events generated (structured data)
3. Classifier scores entity → threat_score + confidence
4. Entity appears on operator map and target queue
5. Operator opens dossier — /score fetched automatically
6. [Optional] Operator clicks Explain → /explain called
7. Backend sends intel report + scores to Claude API
8. LLM returns natural language reasoning + disagreement flag
9. Operator acts: approve / deny / more intel / tactical command
10. Outcome logged for post-scenario debrief

State Architecture

Global State — Zustand Store

One store owns all shared state. No prop drilling between siblings.

- entities[] — live entity list, refreshed every 2s from /api/entities
- simState — simulation metadata, refreshed every 5s from /api/state
- selectedEntityId — currently focused entity, set by map click or queue row click

Server State — TanStack Query

TanStack Query manages all server communication. Polling loops replace all useEffect/setInterval patterns. On success, each query pushes data into the matching Zustand setter.

- /api/entities — refetchInterval: 2000ms
- /api/state — refetchInterval: 5000ms
- /api/entities/{id}/score — fires on entity select (enabled flag)
- /api/entities/{id}/explain — fires on Explain button click (on-demand, not polling)

Local State — DossierPanel

llmResult, llmLoading, and llmError live locally in DossierPanel and reset whenever selectedEntityId changes. The panel never shows stale LLM results from a previous entity.

Tech Stack

Layer	Technology
Simulation & API	Python 3.13, FastAPI, uvicorn
Classifier	scikit-learn, numpy, pandas
LLM Explainer	Claude API (claude-sonnet-4-6), anthropic SDK
Frontend Scaffold	React 19, Vite 6, TypeScript 5

Styling	Tailwind CSS v4
Components	shadcn/ui
Map	MapLibre GL JS + react-map-gl (no API key required)
Map Tiles	Stadia Maps alidade_smooth_dark (free, no credit card)
Server State	TanStack Query v5 — polling, cache invalidation
Client State	Zustand v5 — entities[], simState, selectedEntityId
Charts	Recharts v2 — feature importance BarChart, debrief RadarChart
Animation	Framer Motion v12 — DossierPanel slide-up, queue transitions
Icons	Lucide React
Timers	react-countdown-circle-timer — per-entity countdown, flashes red under 10s
Package Manager	pnpm
Real-time Updates	Polling (TanStack Query) — entities every 2s, sim state every 5s
Data	Synthetic, generated by simulation engine
Environment	Python 3.13, Node 22, venv

Post-Scenario Debrief

After each scenario the operator sees a debrief screen tracking:

- Approve/deny accuracy — how many decisions matched ground truth
- False positive rate — hostiles missed or civilians wrongly approved
- Friendly asset performance — casualties, objectives cleared, efficiency
- AI agreement rate — how often the operator followed the classifier vs. overrode it
- LLM vs. classifier disagreements — cases where the two AI layers conflicted
- Operator bias patterns — e.g. always trusting the AI, always approving under time pressure

Rendered as a Recharts RadarChart with a full decision log table below. Accessible at route /debrief.

Build Order

Each step produces something visible and runnable in the browser. Backend steps 1–3 are complete.

#	Step	Status	Definition of Done
1	**Simulation engine**	✅ Complete	Entity data structures, spawner, movement system
2	**Classifier**	✅ Complete	Synthetic training data, model training, FastAPI exposure
3	**LLM Explainer**	✅ Complete	Claude API wired in for on-demand reasoning

4	**Frontend scaffold + dark theme**	☐ Next	Vite + React 19 + TypeScript, Tailwind dark tokens, layout shell visible
5	**TanStack Query + Zustand**	☐	Polling /api/entities every 2s, store populated, entity count in Header
6	**TargetQueue**	☐	Scrollable queue with colour coding, threat score badges, entity selection
7	**MapView**	☐	MapLibre dark tiles, entity markers at correct lat/lon, heading arrows, marker selection
8	**DossierPanel + ClassifierResult**	☐	Panel slides up on entity select, /score fetched, threat bar and feature importance BarChart
9	**ActionBar + decisions**	☐	Correct buttons per entity type, POST to /decide and /command, status updates on next poll
10	**LLMExplainer**	☐	Explain button triggers /explain, reasoning chain renders, disagreement indicator
11	**Countdown timers**	☐	react-countdown-circle-timer per queue row, flashes red under 10s
12	**Debrief screen**	☐	Route /debrief renders RadarChart, decision log table, AI agreement rate
13	**Polish + deploy**	☐	Scanline overlay, Framer Motion tuning, Vercel deploy, README with screenshot